

Module 06: Test-Driven Development

React + Node ToDo stack



- Build a minimal full-stack ToDo app.
- Use testing to protect behavior while features evolve.
- Study testing from unit checks through acceptance journeys.

Module Objectives

What learners should be able to do



- Understand why Test-Driven Development improves confidence.
- Connect a React frontend to a Node/Express backend.
- Identify the right test type for each reliability question.
- Turn prompts and QA cases into executable test suites.

Application Under Test

Simple enough to learn, real enough to test



- Backend: Node/Express API with in-memory task storage.
- Frontend: Vite React app for task entry and task display.
- Core API routes: GET /tasks, POST /tasks, PATCH /tasks/:id/complete.
- Core user flow: create, list, complete, and reopen a task.

Lab 6.1: ToDo First Pass

Build the working stack

- Run the backend on port 4000.
- Run the frontend through Vite.
- Submit title, due date, and notes from the UI.
- Keep the UI reactive to saved backend state.
- Observe that in-memory data resets when the server restarts.

Lab 6.2: Testing Second Pass

Design coverage across the system



- Use prompts to generate or guide automated test suites.
- Cover unit, integration, end-to-end, frontend, and API testing.
- Add performance, security, regression, smoke, and acceptance checks.
- Clarify inputs, expected results, and tools for each testing layer.

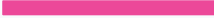
Unit, Integration, And API Tests

Testing logic and contracts

- Unit: validate required task title and toggle completed state.
- Integration: POST a task, then GET it from the backend list.
- Integration: PATCH completion twice and verify final state.
- API: confirm JSON responses, 400 validation errors, and 404 missing IDs.

Frontend And End-To-End Tests

Testing the user experience



- Frontend: required title prevents empty submission.
- Frontend: form clears after a successful save.
- Frontend: useful error appears when backend is unavailable.
- End-to-end: user adds a task, sees it, completes it, and reopens it.

Performance, Security, Regression

Testing reliability under change and pressure



- Performance: measure response time for repeated GET and POST requests.
- Security: reject invalid input and handle script-like values safely.
- Security: large payloads should not crash the server.
- Regression: rerun create, toggle, and validation checks after updates.

Smoke And Acceptance Tests

Fast readiness checks and business-level proof



- Smoke: backend starts and GET /tasks returns 200.
- Smoke: frontend loads without a blank screen.
- Smoke: basic task creation works before deeper testing.
- Acceptance: user can create, view, complete, and reopen a task.
- Acceptance: invalid input gives visible feedback and creates no task.

README.md vs test-cases.md

How the two testing files differ

- README.md is the testing strategy and prompt guide.
- README.md explains each test type and gives sample prompts or tool ideas.
- test-cases.md is the detailed QA test specification.
- test-cases.md gives test IDs, objectives, data, steps, expected results, and priorities.
- Use README.md to plan or generate tests; use test-cases.md to execute or automate them.